# Microservices Ticket Booking System

Tehnička Specifikacija i Arhitektura Samostalnog Backend Projekta

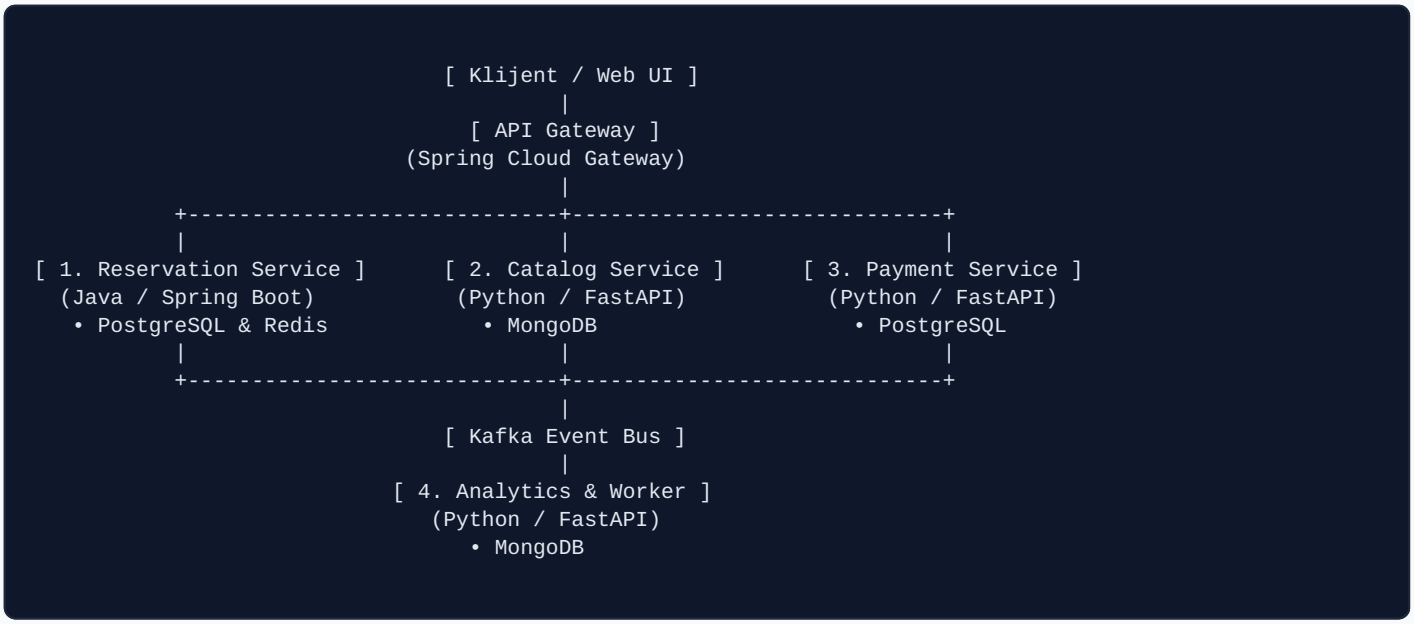
Polyglot Architecture | Java 21 / Spring Boot 3 | Python 3.12 / FastAPI | PostgreSQL & MongoDB | Redis Distributed Lock | Kafka SAGA Pattern

## 1. Pregled i Tehnički Stak

Ovaj sistem je koncipiran kao distribuirana mikroservisna platforma za real-time rezervaciju i kupovinu karata za događaje. Kombinuje visoke performanse, transakcionu konzistentnost i otpornost na veliko opterećenje pri masovnim rezervacijama.

| KOMPONENTA | TEHNOLOGIJA | SVRHA / OPIS |
|---|---|---|
| **API Gateway & Auth** | Spring Cloud Gateway / JWT | Centralna ulazna tačka, ruting zahteva, validacija tokena i Rate Limiting. |
| **Core Service** | Java 21 / Spring Boot 3 | Upravljanje rezervacijama, transakcijama i sprovođenje Redis lock-a. |
| **Python Services** | Python 3.12 / FastAPI | Catalog Service (događaji), Payment Service i Analytics Worker. |
| **Baze podataka** | PostgreSQL & MongoDB | PostgreSQL (ACID transakcije) + MongoDB (fleksibilni katalozi i analitika). |
| **Cache & Lock** | Redis (In-Memory) | Distribuirano zaključavanje sedišta i upravljanje TTL tajmerima (10 min). |
| **Message Broker** | Apache Kafka / RabbitMQ | Asinhrona komunikacija između servisa i SAGA koorganizacija. |

## 2. Arhitektura Servisa



**Detalji Pojedinačnih Servisa:**

### 1. Catalog & Event Service `Python / FastAPI` `MongoDB`

Odgovornost: Upravljanje događajima, izvođačima, dvoranama i rasporedom sedenja.

- **Zašto MongoDB?** Fleksibilna struktura dokumenata (koncert sa numerisanim sedištima, festival po zonama, klupska svirka bez sedenja).
- **Kolekcije:** `events` (naslov, opis, status), `venues` (lokacija, sektori, redovi, kapacitet).

### 2. Inventory & Reservation Service `Java / Spring Boot 3` `PostgreSQL + Redis`

Odgovornost: Najkritičniji transakcioni motor. Praćenje statusa sedišta, privremeno zaključavanje i sprečavanje overbooking-a.

- **Zašto PostgreSQL & Redis?** PostgreSQL obezbeđuje ACID garanciju za konačnu prodaju, dok Redis obezbeđuje milisekundni *Distributed Lock* i TTL za privremenu rezervaciju (10 minuta).
- **Tabele:** `seats` (id, event_id, seat_number, status), `reservations` (id, user_id, expires_at, status).

### 3. Payment Service `Python / FastAPI` `PostgreSQL`

Odgovornost: Obrada finansijskih transakcija i simulacija eksternog Payment Gateway-a.

- **Funkcionalnosti:** Validacija `Idempotency-Key` zaglavlja protiv duple naplate, obrada uplata, emitovanje događaja uspeha/greške na Kafku.

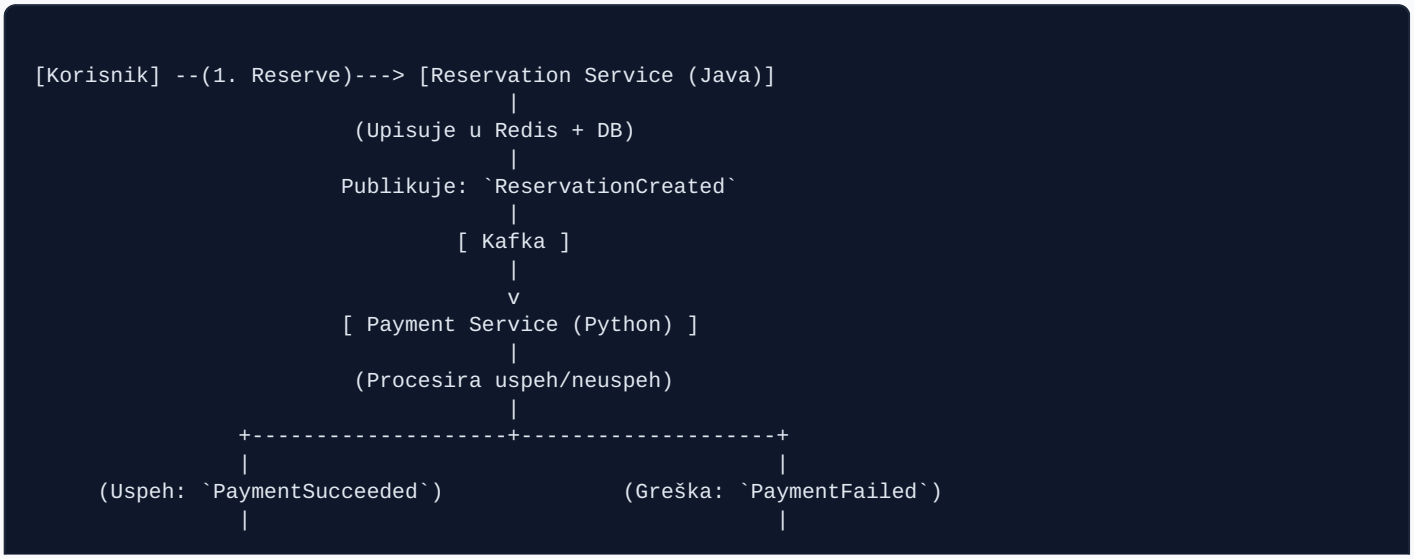
### 4. Analytics & Ticket Worker Service `Python / FastAPI / Celery` `MongoDB`

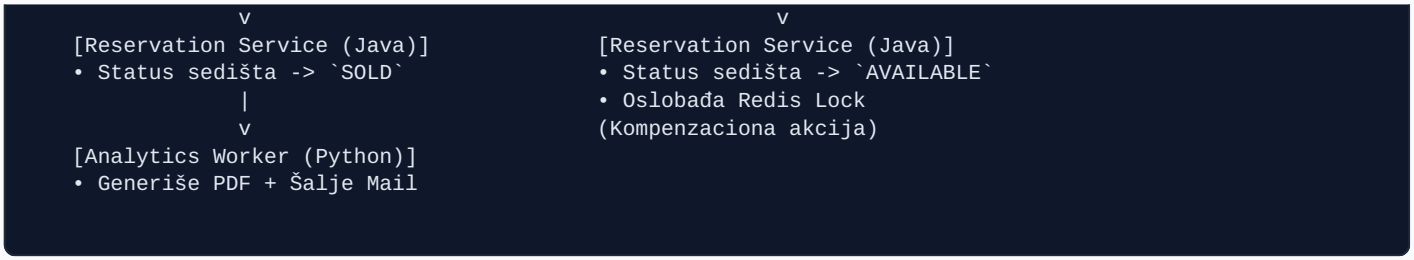
Odgovornost: Asinhrona obrada poruka sa Kafka bus-a.

- **Funkcionalnosti:** Generisanje PDF ulaznice sa QR kodom, slanje obaveštenja korisniku, agregacija analitičkih podataka o prodaji u MongoDB.

## 3. Tok Podataka i SAGA Obrazac (Choreography)

Radi očuvanja konzistentnosti podataka kroz distribuirane servise bez globalnih transakcija, primenjuje se **Choreography SAGA** obrazac uz kompenzacione akcije:






## 4. Mape Puta i Faze Razvoja

**FAZA 1**

**Infrastruktura i MongoDB Catalog Service (Python)**

Podizanje Docker okruženja (PostgreSQL, MongoDB, Redis, Kafka). Izrada Catalog servisa u FastAPI-ju uz Motor asinhroni drajver za upravljanje događajima.

**FAZA 2**

**Core Transactional Engine (Java / Spring Boot)**

Implementacija Reservation servisa. Pisanje logike za rezervaciju sedišta uz Redis distribuirano zaključavanje i pesimističko zaključavanje u PostgreSQL-u.

**FAZA 3**

**Kafka Event Bus & Payment Service (Python)**

Povezivanje servisa preko Kafke. Implementacija Payment servisa i SAGA kompenzacije: pri neuspešnoj uplati vraća se `PaymentFailedEvent` koji oslobađa sedište.

**FAZA 4**

**API Gateway, Testcontainers & Dockerization**

Podešavanje Spring Cloud Gateway-a sa JWT autentifikacijom. Pisanje integracionih testova u Spring Boot-u koristeći Testcontainers za simulaciju celog okruženja.

### 🎯 Vrednost Projekta u Biografiji / Tehničkim Intervjuima

- **Polyglot Microservices:** Praktični dokaz kombinovanja Java (Spring Boot) za enterprise transakcije i Python (FastAPI) za brze asihrone servise.
- **Hibridna Baza Podataka:** Upotreba odgovarajuće baze za odgovarajući problem (PostgreSQL za ACID transakcije, MongoDB za dokumente/analitiku).
- **Visoka Konkurentnost:** Rešavanje race condition problema i overbooking-a pomoću Redis Lock-a pod velikim opterećenjem.
- **Resiliency & SAGA:** Razumevanje distribuiranih transakcija i upravljanje greškama bez narušavanja integriteta sistema.